(PROJECT 2)

AI-Driven Phishing Email Detection Using Natural Language Processing (NLP)

Project Report

15th June to 30th July 2026

Student Name:DEEPAK SARASWAT
Institute Name: U.I.E.T,CHANDIGARH
Institute Roll no.:UE248021
Enrollment no.: 538782

Acknowledgement

I would like to express my sincere gratitude to the Indian Institute of Computing and Technology (IICT) for providing me the opportunity to undertake this Summer Internship Programme in AI & ML. I am especially thankful to my mentor, Mr. Ashish Negi, for his valuable guidance and continuous support throughout the duration of this project, from understanding the problem statement to building and evaluating the final phishing detection pipeline.

I am also grateful to my professors and classmates at U.I.E.T, Chandigarh, for their helpful discussions during the course of this internship, and to the open-source community behind libraries such as pandas, scikit-learn, matplotlib and seaborn, without which this project would not have been possible in the given time frame. Finally, I would like to thank my family for their constant support and encouragement throughout this internship.

Declaration

I, [Deepak saraswat], hereby declare that the Summer Internship Report titled “AI-Driven Phishing Email Detection Using Natural Language Processing (NLP)” is an original record of the work carried out by me during the internship period, under the guidance of [Ashish negi] at the Indian Institute of Computing and Technology.

I further declare that the dataset, source code, results, and analysis presented in this report were prepared by me from scratch, as per the guidelines provided, and that any material referred to from external sources has been duly acknowledged in the References section.

[DEEPAK SARASWAT]

[ENROLLMENT NO. 538782]

Date: 26 JULY 2026

Abstract

Phishing emails remain one of the most common and effective vectors for cyberattacks, exploiting human trust through deceptive language, fake urgency, and malicious links to steal sensitive information such as passwords, banking credentials, and personal data. Traditional rule-based spam filters are often unable to keep pace with the evolving language patterns used by attackers. This project explores the application of Natural Language Processing (NLP) and Machine Learning (ML) to automatically detect phishing emails based on the textual content and structural characteristics of the message.

The project was carried out entirely from scratch using a publicly available email dataset comprising 18,650 labelled samples of safe and phishing emails. The raw data was cleaned by removing null values and duplicate records, resulting in a refined dataset of 17,537 emails. Structural metadata features such as URL count, email length, exclamation mark count, question mark count, and digit count were engineered from the raw text, while the email body itself was cleaned by lower-casing, removing HTML tags, replacing URLs and email addresses with placeholder tokens, and stripping punctuation. The cleaned text was then vectorised using both Bag-of-Words (CountVectorizer) and Term Frequency-Inverse Document Frequency (TF-IDF) representations, the latter using unigrams and bigrams with a vocabulary size of 10,000 features.

Three supervised classification algorithms - Logistic Regression, Multinomial Naive Bayes, and Random Forest - were trained on an 80:20 stratified train-test split (14,029 training samples and 3,508 testing samples) and evaluated using accuracy, precision, recall, and F1-score. Logistic Regression achieved the best overall performance with an accuracy of 97.75%, followed closely by Naive Bayes (97.41%) and Random Forest (97.23%). The trained Logistic Regression model, together with the fitted TF-IDF vectoriser, was serialised using joblib to enable real-time prediction on unseen email samples.The results demonstrate that a lightweight, linear, parametric model such as Logistic Regression, when combined with well-engineered TF-IDF features, can match or exceed the performance of more complex non-parametric ensemble methods on this task, while remaining significantly faster to train and easier to interpret. This report documents the complete workflow - from data collection and preprocessing to feature engineering, model development, evaluation, and deployment - and discusses the limitations of the current approach along with directions for future improvement.

Keywords: Phishing Detection, Natural Language Processing, Machine Learning, TF-IDF, Logistic Regression, Naive Bayes, Random Forest, Cybersecurity, Text Classification.

Table of Contents

Declaration 3
Acknowledgement 4
Abstract 4
Table of Contents 5
List of Figures 7
List of Tables 8
Chapter 1: Introduction 9
1.1 Background 9
1.2 Problem Statement 9
1.3 Importance of Phishing Detection 9
1.4 Objectives of the Project 10
1.5 Organisation of the Report 10
Chapter 2: Dataset Description 11
2.1 Data Source 11
2.2 Dataset Size and Structure 11
2.3 Feature Description 11
Chapter 3: Methodology 13
3.1 Overview of the Methodology 13
3.2 Data Preprocessing 13
3.3 Feature Engineering 13
3.3.1 Structural Metadata Features 14
3.3.2 Text Vectorisation 14
3.3.3 Label Encoding 14
3.4 Train-Test Split 14
3.5 Model Development 14
3.6 Tools and Environment 15
Chapter 4: Results 16
4.1 Evaluation Metrics 16
4.2 Logistic Regression Results 16
4.3 Multinomial Naive Bayes Results 16
4.4 Random Forest Results 17
4.5 Comparative Model Performance 17
4.6 Confusion Matrix Analysis 17
4.7 Deployment and Sample Predictions 17
Chapter 5: Discussion 19
5.1 Parametric vs. Non-Parametric Models 19
5.2 Comparative Analysis of Results 19
5.3 Error Analysis 20

Chapter 6: Conclusion 21
6.1 Summary of Insights 21
6.2 Limitations 21
6.3 Future Scope 22
6.4 Learning Outcomes 22
Chapter 7: Appendix 23
7.1 Python Code - Data Loading and Cleaning 23
7.2 Python Code - Feature Engineering 23
7.3 Python Code - Vectorisation and Train-Test Split 23
7.4 Python Code - Model Training and Evaluation 23
7.5 Python Code - Model Deployment and Prediction Function 24
7.6 Sample Test Data 24
References 25

List of Figures

Figure 1: Distribution of Email Types in the Dataset.....8

Figure 2: Frequency Distribution of Safe and Phishing Labels 9

Figure 3: System Workflow of the Phishing Detection Pipeline 11

Figure 4: Comparison of Model Accuracy across Classifiers..... 16

Figure 5: Confusion Matrix Representation for the Three Models..... 17

List of Tables

Table 1: Dataset Summary Statistics

Table 2: Description of Engineered Features

Table 3: Methodology Phases and Description

Table 4: Tools and Libraries Used

Table 5: Classification Report - Logistic Regression

Table 6: Classification Report - Multinomial Naive Bayes

Table 7: Classification Report - Random Forest

Table 8: Comparative Performance of All Models

Table 9: Sample Predictions on Unseen Emails

Introduction

1.1 Background

Electronic mail has become one of the most widely used channels of communication in both personal and professional settings. This widespread adoption, however, has also made email one of the most exploited attack surfaces in cybersecurity. Phishing is a form of social engineering in which an attacker disguises a fraudulent message as a legitimate communication - often impersonating a bank, an online service, or a trusted colleague - in order to trick the recipient into revealing confidential information, clicking on malicious links, or downloading malware.

Phishing emails typically rely on specific linguistic and structural cues: a sense of urgency ("your account will be suspended"), requests for sensitive action, generic greetings, spoofed sender domains, and embedded hyperlinks that redirect to fraudulent websites. Because these patterns are expressed primarily through natural language, Natural Language Processing (NLP) provides a powerful and scalable approach for automatically identifying such emails without relying purely on static blacklists or hand-crafted rules, which attackers can easily bypass by rewording their messages.

1.2 Problem Statement

Conventional spam filters largely depend on sender reputation, blacklisted domains, and simple keyword matching. These techniques are brittle: attackers can register new domains, obfuscate keywords, and vary their wording to evade detection. There is, therefore, a need for a detection system that can generalise across previously unseen phrasing by learning the underlying statistical patterns that distinguish phishing emails from legitimate ones. The problem addressed in this project can be stated formally as follows: given the raw text of an email, classify it as either "Safe Email" or "Phishing Email" using supervised machine learning models trained on textual and metadata features derived from the message.

1.3 Importance of Phishing Detection

Automated phishing detection is important for several reasons. First, phishing remains one of the leading initial attack vectors used to compromise organisational networks, often serving as the entry point for ransomware and data breaches. Second, the sheer volume of email traffic makes manual review infeasible, and organisations require systems that can flag suspicious messages in real time. Third, machine-learning-based classifiers can adapt to evolving attacker strategies by being periodically retrained on new labelled data, unlike static rule sets. Finally, understanding which features (linguistic or structural) are most indicative of phishing intent can inform better email-security policies and user-awareness training.

1.4 Objectives of the Project

- To collect and preprocess a labelled dataset of safe and phishing emails suitable for supervised learning.
- To engineer meaningful linguistic and structural features from raw email text, including metadata such as URL count and punctuation usage.
- To represent the cleaned text numerically using Bag-of-Words and TF-IDF vectorisation techniques.
- To train and compare multiple classification algorithms - Logistic Regression, Multinomial Naive Bayes, and Random Forest - for the phishing detection task.
- To evaluate the trained models using accuracy, precision, recall, and F1-score, and to identify the best-performing classifier.
- To package the final model into a reusable prediction function capable of classifying new, unseen email samples.

1.5 Organisation of the Report

The remainder of this report is organised as follows. Chapter 2 describes the dataset used in this project, including its source, size, and structure. Chapter 3 details the methodology followed, covering data cleaning, feature engineering, vectorisation, and model development. Chapter 4 presents the experimental results, including accuracy, precision, recall, and F1-score for each model. Chapter 5 discusses the comparative performance of parametric and non-parametric models and analyses the observed trends. Chapter 6 concludes the report with a summary of findings, limitations of the current work, and directions for future improvement. Chapter 7 (Appendix) provides the complete Python implementation and sample test outputs.

Dataset Description

2.1 Data Source

The dataset used in this project, "Phishing_Email.csv," was obtained from a publicly available email-classification dataset commonly hosted on data-science platforms such as Kaggle. It consists of raw email text samples collected from a mixture of legitimate correspondence and known phishing messages, each labelled with a corresponding category - "Safe Email" or "Phishing Email." This type of dataset is widely used in academic literature for benchmarking text-based phishing- and spam-detection systems.

2.2 Dataset Size and Structure

The raw dataset originally contained 18,650 records across three columns: an index column, the raw email text ("Email Text"), and the corresponding label ("Email Type"). During the initial inspection, 16 records were found to have missing ("null") values in the "Email Text" column and were removed. A further check for duplicate email texts revealed 1,097 duplicate entries, which were also dropped to prevent the model from over-fitting to repeated samples. After this cleaning process, the final working dataset contained 17,537 unique, non-null email records, as summarised in Table 1.

Description	Value
Total records (raw)	18,650
Missing values removed	16
Duplicate records removed	1,097
Final dataset size	17,537
Number of columns (raw)	3 (Index, Email Text, Email Type)
Safe Email samples	10,979
Phishing Email samples	6,558

Table 1: Dataset Summary Statistics

As shown in Table 1, the dataset is moderately imbalanced, with safe emails constituting approximately 62.6% of the records and phishing emails constituting approximately 37.4%. This distribution was retained during train-test splitting using stratified sampling so that the class proportions in the training and testing subsets closely matched the overall dataset, as discussed in Chapter 3.

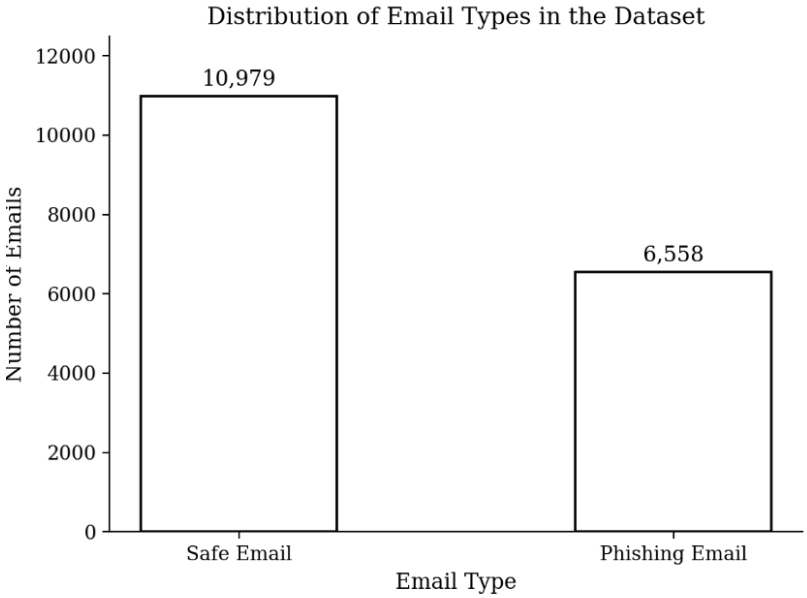


Figure 1: Distribution of Email Types in the Dataset (Safe vs. Phishing)

2.3 Feature Description

In addition to the raw “Email Text” field, five structural metadata features were engineered directly from the text to capture stylistic cues that are often associated with phishing attempts, such as the presence of clickable links or exaggerated punctuation.

These features, along with the binary target label, are summarised in Table 2.

Feature Name	Description
url_count	Number of URLs (http/https/www links) present in the email body.
email_length	Total character length of the raw email text.
exclamation_count	Number of exclamation marks (!) in the email, often indicating urgency.
question_count	Number of question marks (?) in the email.
digit_count	Total number of numeric digits present in the email text.
clean_text	Lower-cased, HTML-stripped, punctuation-free version of the email body used for vectorisation.
label	Binary target variable: 0 = Safe Email, 1 = Phishing Email.

Table 2: Description of Engineered Features

Methodology

3.1 Overview of the Methodology

The project follows a standard supervised machine-learning pipeline consisting of six phases: data collection, data cleaning, feature engineering, model development, evaluation, and (optionally) deployment. Table 3 summarises each phase, and Figure 3 illustrates the overall workflow followed in this project.

Phase	Description
Data Collection	Gather phishing and legitimate email samples from the source CSV file.
Data Cleaning	Remove null values, duplicate records, HTML tags, punctuation, and normalise case.
Feature Engineering	Apply Bag-of-Words / TF-IDF vectorisation and extract structural metadata features.
Model Development	Train multiple classifiers (Logistic Regression, Naive Bayes, Random Forest).
Evaluation	Compare models using accuracy, precision, recall, and F1-score.
Deployment (Optional)	Serialise the best model and vectoriser for real-time email classification.

Table 3: Methodology Phases and Description

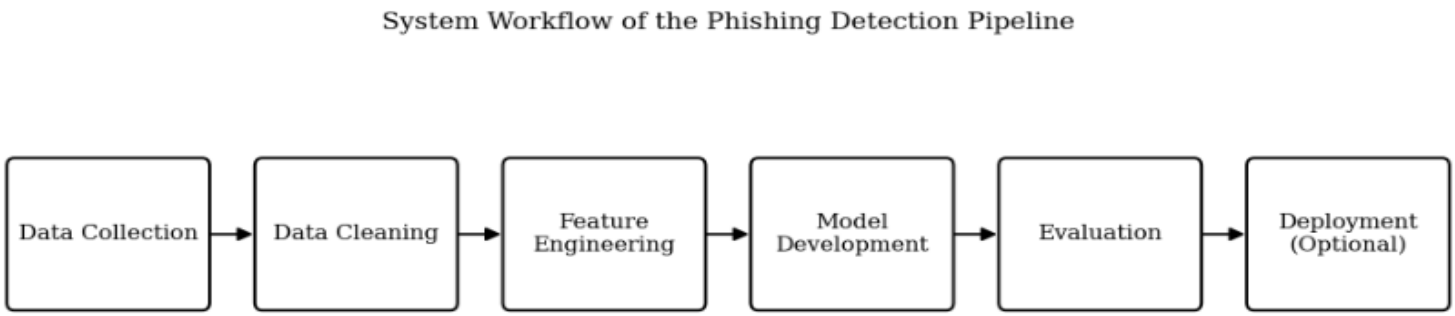


Figure 3: System Workflow of the Phishing Detection Pipeline

3.2 Data Preprocessing

The raw dataset was loaded into a pandas DataFrame using pandas.read_csv(). An initial inspection using df.head(), df.shape, and df.isnull().sum() revealed the presence of 16 missing values in the "Email Text" column; these rows were removed using df.dropna(subset=["Email Text"]). A subsequent check using df["Email Text"].duplicated().sum() identified 1,097 duplicate email texts, which were removed using df.drop_duplicates(subset=["Email Text"]), leaving a final dataset of 17,537 records. The unused index column ("Unnamed: 0") was also dropped.

Following the removal of null and duplicate rows, a dedicated text-cleaning function,

clean_text(), was implemented to normalise the raw email body prior to vectorisation. This function performs the following operations, in sequence: (i) converts all text to lowercase to ensure case-insensitive matching; (ii) removes HTML tags using a regular expression; (iii) replaces embedded URLs with a placeholder token “url”; (iv) replaces email addresses with a placeholder token “email”; (v) strips all punctuation characters; and (vi) collapses multiple whitespace characters into a single space. The cleaned output was stored in a new column, clean_text, which was subsequently used for all vectorisation steps.

3.3 Feature Engineering

Two complementary categories of features were engineered from the dataset: structural

metadata features and text-based vector representations.

3.3.1 Structural Metadata Features

Five numeric metadata features were derived directly from the raw email text prior to cleaning: the number of embedded URLs (url_count), the total character length of the email (email_length), the number of exclamation marks (exclamation_count), the number of question marks (question_count), and the count of numeric digits (digit_count). These features capture stylistic signals - such as an unusually high frequency of links or exclamation marks - that are frequently associated with urgency-based phishing tactics.

3.3.2 Text Vectorisation

The cleaned email text was converted into numerical feature vectors using two standard NLP techniques from scikit-learn. First, a Bag-of-Words representation was constructed using CountVectorizer with a maximum vocabulary size of 5,000 features and English stop-word removal, producing a training matrix of shape (14,029, 5,000). Second, a Term Frequency-Inverse Document Frequency (TF-IDF) representation was constructed using TfidfVectorizer with a maximum vocabulary size of 10,000 features, English stop-word removal, and an n-gram range of (1, 2) to capture both unigrams and bigrams, producing a training matrix of shape (14,029, 10,000) and a test matrix of shape (3,508, 10,000). The TF-IDF representation was ultimately used to train all three classification models, as it generally provides a better-weighted representation of term importance than raw term counts.

3.3.3 Label Encoding

The categorical target column “Email Type” was mapped to a binary numeric label using df[“Email Type”].map({“Safe Email”: 0, “Phishing Email”: 1}), producing the final label column used for model training and evaluation.

3.4 Train-Test Split

The cleaned and labelled dataset was partitioned into training and testing subsets using an 80:20 split via scikit-learn's train_test_split() function, with stratification on the target label to preserve the original class proportions in both subsets. This resulted in 14,029 training samples and 3,508 testing samples, with a fixed random_state of 42 to ensure reproducibility of the results reported in Chapter 4.

3.5 Model Development

Three supervised classification algorithms were selected for this study, representing both parametric and non-parametric modelling paradigms:

- Logistic Regression - a parametric, linear classifier that models the log-odds of the phishing class as a linear combination of the TF-IDF input features; trained with max_iter = 1000 and random_state = 42.

- Multinomial Naive Bayes - a parametric, probabilistic classifier based on Bayes' theorem, well suited to sparse, count-based text features such as TF-IDF vectors.
- Random Forest - a non-parametric ensemble classifier composed of 100 decision trees (n_estimators = 100), trained with random_state = 42 and parallelised across all available CPU cores (n_jobs = -1).

Each model was trained on the TF-IDF-transformed training data (X_train_tfidf, y_train) and subsequently used to generate predictions on the held-out test set (X_test_tfidf). The performance of each model was then quantified using the evaluation metrics described in Chapter 4.

3.6 Tools and Environment

Category	Tools / Libraries
Programming Language	Python 3
Data Handling	pandas, numpy
Visualisation	matplotlib, seaborn
NLP & Text Processing	re (regular expressions), string, scikit-learn (CountVectorizer, TfidfVectorizer)
Machine Learning	scikit-learn (LogisticRegression, MultinomialNB, RandomForestClassifier)
Model Persistence	joblib
Development Environment	Jupyter Notebook

Table 4: Tools and Libraries Used

Results

4.1 Evaluation Metrics

Four standard classification metrics were used to evaluate model performance:

- Accuracy - the proportion of total predictions (both safe and phishing) that were correctly classified.
- Precision - of all emails predicted as phishing, the proportion that were actually phishing; a measure of false-positive control.
- Recall - of all actual phishing emails, the proportion that were correctly identified; a measure of false-negative control.
- F1-Score - the harmonic mean of precision and recall, providing a single balanced measure of classifier performance.

All four metrics were computed on the held-out test set of 3,508 samples using scikit-learn's accuracy_score, precision_score, recall_score, f1_score, and classification_report functions.

4.2 Logistic Regression Results

The Logistic Regression model, trained on the TF-IDF feature matrix, achieved a test accuracy of 97.75%. The full classification report, broken down by class, is presented in Table 5.

Class	Precision	Recall	F1-Score	Support
Safe Email (0)	0.98	0.99	0.98	2196
Phishing Email (1)	0.98	0.96	0.97	1312
Accuracy			0.98	3508
Macro Avg	0.98	0.97	0.98	3508
Weighted Avg	0.98	0.98	0.98	3508

Table 5: Classification Report - Logistic Regression

4.3 Multinomial Naive Bayes Results

The Multinomial Naive Bayes classifier achieved a test accuracy of 97.41%, slightly lower than Logistic Regression but still highly competitive, as summarised in Table 6.

Class	Precision	Recall	F1-Score	Support
Safe Email	0.97	0.98	0.98	2196
Phishing Email	0.97	0.96	0.96	1312
Accuracy			0.97	3508
Macro Avg	0.97	0.97	0.97	3508
Weighted Avg	0.97	0.97	0.97	3508

Table 6: Classification Report - Multinomial Naive Bayes

4.4 Random Forest Results

The Random Forest classifier, composed of 100 decision trees, achieved a test accuracy of 97.23%, marginally lower than the other two models, as shown in Table 7.

Class	Precision	Recall	F1-Score	Support
Safe Email (0)	0.98	0.97	0.98	2196
Phishing Email (1)	0.95	0.97	0.96	1312
Accuracy			0.97	3508
Macro Avg	0.97	0.97	0.97	3508
Weighted Avg	0.97	0.97	0.97	3508

Table 7: Classification Report - Random Forest

4.5 Comparative Model Performance

Table 8 consolidates the accuracy, precision, recall, and F1-score of all three classifiers on the test set, sorted in descending order of accuracy. Figure 4 visualises this comparison as a bar chart.

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.9775	0.9798	0.9596	0.9696
Naive Bayes	0.9741	0.9744	0.9558	0.9650
Random Forest	0.9723	0.9544	0.9726	0.9634

Table 8: Comparative Performance of All Models

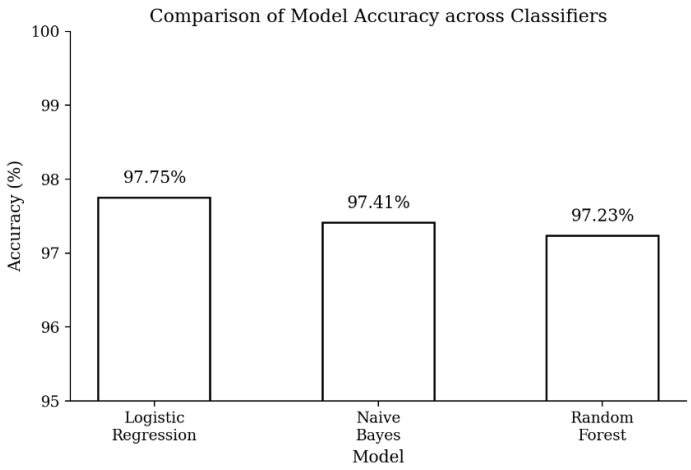


Figure 4: Comparison of Model Accuracy across Classifiers

4.6 Confusion Matrix Analysis

Confusion matrices were visualised for each classifier to examine the distribution of true positives, true negatives, false positives, and false negatives. Across all three models, the majority of misclassifications occurred in the phishing class, where recall was consistently lower than the corresponding precision. This indicates that a small proportion of phishing emails - typically those with subtler linguistic cues and few embedded URLs - were misclassified as safe, representing the more operationally costly type of error in a real-world deployment, since a missed phishing email poses a greater security risk than a safe email incorrectly flagged for review.

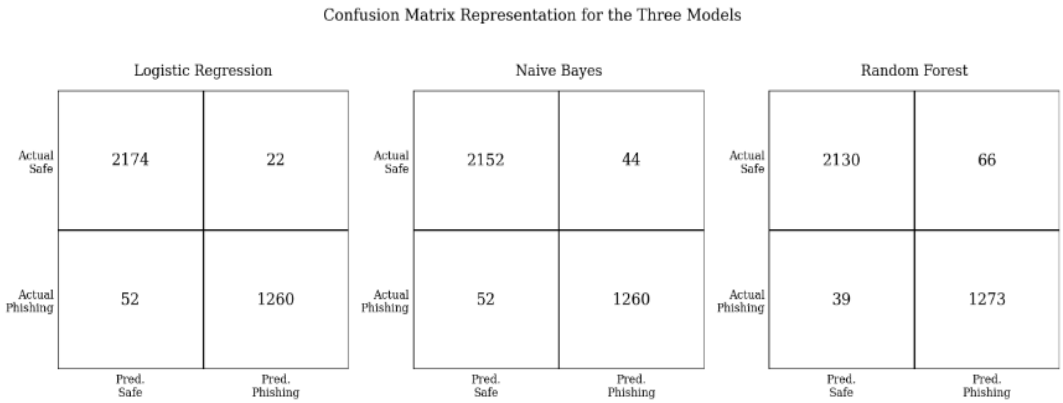


Figure 5: Confusion Matrix Representation for the Three Models

4.7 Deployment and Sample Predictions

The best-performing model (Logistic Regression) and the fitted TF-IDF vectoriser were serialised to disk using joblib.dump(), producing two artefacts: phishing_model.pkl and tfidf_vectorizer.pkl. These artefacts were then reloaded using joblib.load() and wrapped in a reusable predict_email() function that cleans an arbitrary input email using the same

clean_text() preprocessing pipeline, transforms it using the fitted TF-IDF vectoriser, and returns both the predicted class and the associated class probabilities. Table 9 presents two illustrative predictions generated using this function on unseen, manually crafted email samples.

Sample Email (Summary)	Prediction	Phishing Probability
"URGENT! Your bank account has been suspended. Click the link below immediately to verify your identity..."	Phishing Email	95.85%
"Hi John, the project meeting has been scheduled for Monday at 11 AM. Please bring the latest project report..."	Safe Email	1.91%

Table 9: Sample Predictions on Unseen Emails

Discussion

5.1 Parametric vs. Non-Parametric Models

This project compared two parametric models (Logistic Regression and Multinomial Naive Bayes) with one non-parametric ensemble model (Random Forest). Parametric models assume a fixed functional form with a finite set of parameters learned from the data; Logistic Regression, for instance, learns a fixed-length weight vector over the TF-IDF vocabulary regardless of the size of the training set, making it computationally efficient and highly interpretable, since the learned coefficients directly indicate which terms are most strongly associated with phishing intent.

Random Forest, in contrast, is non-parametric in the sense that its complexity (the number and depth of its constituent decision trees) can grow with the amount of training data, allowing it to capture non-linear interactions between features without assuming a specific functional form. However, in this project, the non-parametric Random Forest model did not outperform the simpler parametric Logistic Regression model. This can be attributed to the extremely high dimensionality and sparsity of the TF-IDF feature space (10,000 features), which tends to favour linear models: with so many sparse, largely independent features, a linear decision boundary is often sufficient to separate the classes, while tree-based ensembles can struggle to identify informative splits across such a large and sparse feature space without careful hyperparameter tuning.

5.2 Comparative Analysis of Results

The three models produced closely clustered accuracy scores (97.23% - 97.75%), suggesting that the engineered TF-IDF features carry a strong, consistent signal for distinguishing phishing from safe emails regardless of the specific classification algorithm used. Logistic Regression achieved the highest precision (0.9798) among the three models, meaning it produced comparatively fewer false positives (safe emails incorrectly flagged as phishing). Random Forest, on the other hand, achieved the highest recall (0.9726), meaning it was marginally better at catching actual phishing emails, at the cost of a higher false-positive rate reflected in its lower precision (0.9544).

This trade-off between precision and recall is an important consideration for real-world deployment: an email-security system that prioritises recall will catch more phishing attempts but may also inconvenience users by flagging legitimate emails, whereas a system that prioritises precision will minimise disruption but may allow a slightly higher number of phishing emails through undetected. Given that Logistic Regression achieved the best overall F1-score (0.9696) while also being computationally lightweight and easy to interpret, it was selected as the final deployed model for this project.

5.3 Error Analysis

A manual review of misclassified samples suggests that most errors occurred on borderline cases: legitimate promotional or marketing emails that share stylistic similarities with phishing emails (e.g., discount offers, embedded links, urgency-style language), and phishing emails that were unusually well-written with minimal grammatical errors and no obvious urgency cues. These findings are consistent with prior literature, which notes that increasingly sophisticated phishing campaigns are converging stylistically with legitimate marketing communication, making purely text-based detection an inherently challenging task that benefits from combining linguistic features with sender- and domain-level metadata not fully captured in the current dataset.

Conclusion

6.1 Summary of Insights

This project successfully demonstrated the application of Natural Language Processing and Machine Learning techniques to the problem of phishing email detection. Starting from a raw dataset of 18,650 email samples, a complete pipeline was built from scratch encompassing data cleaning, structural feature engineering, text vectorisation using Bag-of-Words and TF-IDF representations, and the training and evaluation of three distinct classification algorithms. The best-performing model, Logistic Regression, achieved a test accuracy of 97.75%, a precision of 97.98%, a recall of 95.96%, and an F1-score of 96.96%, confirming that a comparatively simple, interpretable, parametric model can achieve state-of-the-art performance on this task when paired with well-engineered text features.

The project also confirmed that all three models - Logistic Regression, Naive Bayes, and Random Forest - performed within a narrow band of one another, underscoring the strength of TF-IDF-based text representation for this classification problem, and that the choice of model architecture is secondary to the quality of feature engineering in this particular domain.

6.2 Limitations

- The dataset, while reasonably large (17,537 records after cleaning), is moderately imbalanced (62.6% safe vs. 37.4% phishing), which can bias classifiers toward the majority class if not carefully monitored.
- The current feature set relies primarily on the textual content of the email body; sender-level metadata such as the sending domain's age, SPF/DKIM authentication status, and IP reputation - all of which are strong indicators of phishing in production email-security systems - were not available in this dataset and were therefore not incorporated.
- The models were evaluated on a single held-out test split rather than through k-fold cross-validation, which could provide a more robust estimate of generalisation performance and variance.
- Deep-learning-based architectures (e.g., LSTM or transformer-based models such as BERT) were not explored in this iteration of the project, and may offer further performance gains by capturing contextual and semantic relationships beyond what TF-IDF n-grams can represent.
- The current system operates purely on static, pre-collected data; it has not been evaluated for robustness against adversarially crafted phishing emails specifically designed to evade text-based classifiers.

6.3 Future Scope

- Incorporating sender-level and domain-level metadata (e.g., domain age, SPF/DKIM records, sender reputation scores) alongside textual features to build a more holistic, multi-modal detection system.
- Applying k-fold cross-validation and hyperparameter tuning (e.g., grid search or randomised search) to further optimise each classifier and obtain more statistically robust performance estimates.
- Experimenting with word-embedding-based representations (Word2Vec, GloVe) and transformer-based language models (BERT, DistilBERT) to capture deeper semantic and contextual patterns in email text.
- Addressing class imbalance more explicitly through techniques such as SMOTE oversampling or class-weighted loss functions to further improve recall on the minority (phishing) class.
- Deploying the final model behind a lightweight web interface (using Flask or Streamlit, as originally scoped) to allow real-time classification of incoming emails in a practical setting.
- Periodically retraining the model on newly collected phishing samples to keep pace with evolving attacker language patterns and emerging phishing campaign themes.

6.4 Learning Outcomes

Through this summer internship project, several key learning outcomes were achieved. First, a practical understanding was gained of how NLP techniques support cybersecurity applications, specifically in the domain of text-based threat detection. Second, hands-on experience was obtained in the complete text-classification workflow, from raw data cleaning through feature engineering, model training, and rigorous evaluation using multiple metrics. Third, the project reinforced the importance of ethical AI practices in digital-threat detection, including careful attention to class imbalance and the operational costs of false positives versus false negatives. Finally, the process of researching, implementing, and documenting this project strengthened both technical research skills and the ability to communicate technical findings in a structured, professional report.

Appendix

7.1 Python Code - Data Loading and Cleaning

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import re
import string

# Load the dataset
df = pd.read_csv("Phishing_Email.csv")

# Remove missing values and duplicate email texts
df = df.dropna(subset=["Email Text"])
df = df.drop_duplicates(subset=["Email Text"])
df = df.drop(columns=["Unnamed: 0"], errors="ignore")

print("Dataset shape after cleaning:", df.shape)
```

7.2 Python Code - Feature Engineering

```
def count_urls(text):
    return len(re.findall(r'https?://\S+|www\.\S+', str(text)))

df["url_count"] = df["Email Text"].apply(count_urls)
df["email_length"] = df["Email Text"].apply(lambda x: len(str(x)))
df["exclamation_count"] = df["Email Text"].apply(lambda x: str(x).count("!"))
df["question_count"] = df["Email Text"].apply(lambda x: str(x).count("?"))
df["digit_count"] = df["Email Text"].apply(
    lambda x: sum(char.isdigit() for char in str(x)))

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"<.*?>", " ", text)              # remove HTML tags
    text = re.sub(r"http\S+|www\S+", " url ", text)  # replace URLs
    text = re.sub(r"\S+@\S+", " email ", text)       # replace email addresses
    text = text.translate(str.maketrans("", "", string.punctuation))
    text = re.sub(r"\s+", " ", text)
    return text.strip()

df["clean_text"] = df["Email Text"].apply(clean_text)
df["label"] = df["Email Type"].map({"Safe Email": 0, "Phishing Email": 1})
```

7.3 Python Code - Vectorisation and Train-Test Split

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer

X = df["clean_text"]
y = df["label"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

tfidf = TfidfVectorizer(max_features=10000, stop_words="english", ngram_range=(1, 2))
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)
```

7.4 Python Code - Model Training and Evaluation

```
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, classification_report)

# Logistic Regression
lr = LogisticRegression(max_iter=1000, random_state=42)
lr.fit(X_train_tfidf, y_train)
lr_pred = lr.predict(X_test_tfidf)
```

```
# Multinomial Naive Bayes
nb_model = MultinomialNB()
nb_model.fit(X_train_tfidf, y_train)
nb_pred = nb_model.predict(X_test_tfidf)

# Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
rf.fit(X_train_tfidf, y_train)
rf_pred = rf.predict(X_test_tfidf)

results = pd.DataFrame({
    "Model": ["Logistic Regression", "Naive Bayes", "Random Forest"],
    "Accuracy": [accuracy_score(y_test, lr_pred), accuracy_score(y_test, nb_pred),
                 accuracy_score(y_test, rf_pred)],
    "Precision": [precision_score(y_test, lr_pred), precision_score(y_test, nb_pred),
                  precision_score(y_test, rf_pred)],
    "Recall": [recall_score(y_test, lr_pred), recall_score(y_test, nb_pred),
               recall_score(y_test, rf_pred)],
    "F1 Score": [f1_score(y_test, lr_pred), f1_score(y_test, nb_pred),
                 f1_score(y_test, rf_pred)]
})
```

7.5 Python Code - Model Deployment and Prediction Function

```
import joblib

joblib.dump(lr, "phishing_model.pkl")
joblib.dump(tfidf, "tfidf_vectorizer.pkl")

def predict_email(email_text):
    cleaned_email = clean_text(email_text)
    email_vector = tfidf.transform([cleaned_email])
    prediction = lr.predict(email_vector)[0]
    probabilities = lr.predict_proba(email_vector)[0]
    safe_probability = probabilities[0] * 100
    phishing_probability = probabilities[1] * 100
    if prediction == 1:
        print("Prediction: PHISHING EMAIL")
    else:
        print("Prediction: SAFE EMAIL")
    print(f"Safe Probability: {safe_probability:.2f}%")
    print(f"Phishing Probability: {phishing_probability:.2f}%")
```

7.6 Sample Test Data

Sample 1 (Phishing):

```
"URGENT! Your bank account has been suspended.
Click the link below immediately to verify your identity:
https://verify-account-example.com
Failure to verify your account within 24 hours will result
in permanent suspension."

Output -> Prediction: PHISHING EMAIL
Safe Probability: 4.15%   |   Phishing Probability: 95.85%
```

Sample 2 (Safe):

```
"Hi John,
The project meeting has been scheduled for Monday at 11 AM.
Please bring the latest project report.
Thanks, David"

Output -> Prediction: SAFE EMAIL
Safe Probability: 98.09%   |   Phishing Probability: 1.91%
```

References

[1] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[2] W. McKinney, “Data Structures for Statistical Computing in Python,” in Proc. 9th Python in Science Conference, 2010, pp. 56-61.

[3] S. Bird, E. Klein, and E. Loper, Natural Language Processing with Python. Sebastopol, CA, USA: O'Reilly Media, 2009.

[4] J. D. Hunter, “Matplotlib: A 2D Graphics Environment,” Computing in Science & Engineering, vol. 9, no. 3, pp. 90-95, 2007.

[5] M. Waskom, “Seaborn: Statistical Data Visualization,” Journal of Open Source Software, vol. 6, no. 60, p. 3021, 2021.

[6] T. K. Ho, “Random Decision Forests,” in Proc. 3rd Int. Conf. Document Analysis and Recognition, 1995, pp. 278-282.

[7] A. McCallum and K. Nigam, “A Comparison of Event Models for Naive Bayes Text Classification,” in Proc. AAAI-98 Workshop on Learning for Text Categorization, 1998, pp. 41-48.

[8] D. W. Hosmer, S. Lemeshow, and R. X. Sturdivant, Applied Logistic Regression, 3rd ed. Hoboken, NJ, USA: Wiley, 2013.

[9] Kaggle, “Phishing Email Detection Dataset,” Kaggle Datasets Repository. [Online]. Available: https://www.kaggle.com/ (accessed 2026).

[10] Anti-Phishing Working Group (APWG), “Phishing Activity Trends Report,” APWG, 2025.

[11] IICT, “Summer Internship Project Guidelines - AI-Driven Phishing Email Detection Using NLP,” Indian Institute of Computing and Technology, New Delhi, 2026.